

Universidade de São Paulo
Instituto de Matemática, Estatística e Ciência da Computação

OBI 2025 - Fase 1 - Nível Sênior

Editorial

Autores: Gabriela Kwong Yamaguti

Explicações dos Problemas

Festa Junina

Tags: Vetores

Perceba que como a escola, o supermercado e a lojinha estão localizados em uma mesma rua reta, podemos modelar suas posições como pontos coordenados em um eixo unidimensional.

Vemos que independentemente da posição inicial da escola, o trajeto ótimo consiste em ir do prédio localizado no extremo esquerdo (a menor coordenada) até o prédio localizado no extremo direito (a maior coordenada) e, depois, realizar o caminho inverso para retornar ao ponto de partida.

Qualquer prédio intermediário será visitado naturalmente durante esse percurso, sem a necessidade de desvios que aumentem a distância. Logo, o cálculo do deslocamento total depende apenas da amplitude máxima entre os pontos mais distantes (a maior e menor coordenada).

Note que na solução ao invés de testar cada variável para achar o máximo e o mínimo, podemos colocar as 3 variáveis em um vetor e ordená-lo. A maior coordenada será o último elemento do vetor e o menor será o primeiro.

Cafeteria

Tags: Matemática

Olhando para o problema, a xícara precisa ser totalmente preenchida. Como o tamanho da xícara é fixo, a quantidade de café que colocamos define automaticamente a quantidade de leite que vai caber. Dessa forma, em vez de pensar no leite, podemos olhar para o problema pelo lado do café.

Conseguimos descobrir o máximo de café permitido pegando o tamanho da xícara e tirando o mínimo de leite exigido. Da mesma forma, o mínimo de café permitido é o tamanho da xícara menos o máximo de leite que o cliente aceita. Assim, nosso único

objetivo é descobrir se existe alguma quantidade de doses de café que termine em um valor dentro desse intervalo.

Para testar isso no código, podemos usar a divisão: quando dividimos o café máximo pelo tamanho da dose (d) usando variáveis inteiras, o C++ descarta os números quebrados. Se pegarmos esse resultado e multiplicamos de volta por d , encontramos o maior valor exato de café que a máquina consegue fazer sem passar do limite máximo.

Agora, basta testar se esse valor também é maior ou igual ao mínimo de café que precisávamos. Se for, o pedido é possível.

Fila

Tags: Vetores

A mesa do professor está logo após a cadeira N . Isso significa que a visão dele em direção ao fundo da sala (cadeira 1) funciona de trás para frente. Um aluno só será bloqueado se houver alguém mais alto ou do mesmo tamanho entre ele e a mesa do professor.

Em vez de testar cada aluno contra todos os outros da frente, a estratégia ideal é começar olhando do aluno mais perto do professor e ir caminhando em direção ao fundo da sala, guardando sempre qual foi a maior altura encontrada até aquele momento.

Para isso criamos uma variável para guardar a maior altura vista (**maior**), começando pelo último aluno da fila. Depois, usamos um laço de repetição que percorre a fila de trás para frente.

Se o aluno atual for menor ou igual ao maior que já registramos, significa que ele está escondido atrás de alguém e não pode ser visto; logo, aumentamos nosso contador de respostas. Caso ele seja estritamente maior, o professor consegue enxergá-lo, e esse aluno passa a ser o novo maior que bloqueará os próximos estudantes mais ao fundo.

Gráfico de Barras

Tags: Matrizes

O maior desafio é que o gráfico de barras deve ser impresso de cima para baixo (linha por linha), porém as barras começam de baixo para cima em colunas.

Para resolver isso, a estratégia ideal consiste em primeiro descobrir qual é a maior altura (H) do gráfico, que corresponde ao brinquedo mais popular, e depois desenhar o gráfico linha por linha, começando do topo (altura H) e descendo até a base (altura 1).

Para fazer isso, ao fazermos a leitura salvamos os valores em um vetor e também, registramos o maior valor (H) encontrado.

Em seguida, usamos dois laços de repetição aninhados.

- O laço de fora representa as linhas do gráfico e faz uma contagem regressiva, começando de **maior** e indo até 1.
- O laço de dentro passa por cada brinquedo do vetor.

Se a popularidade do brinquedo atual for maior ou igual à linha que estamos desenhando, significa que a barra já atingiu aquela altura, então imprimimos 1. Caso contrário, aquela posição ainda deve ser preenchida com um espaço vazio, então imprimimos 0.

Implementações

Festa Junina

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int32_t main()
5  {
6      int p[3];
7      cin >> p[0] >> p[1] >> p[2];
8      sort(p, p + 3);
9      cout << (p[2] - p[0]) * 2;
10 }
```

Cafeteria

```
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int32_t main()
5  {
6      int a, b, c, d;
7      cin >> a >> b >> c >> d;
8      int maxi = c - a, mini = c - b;
9      if ((maxi / d) * d >= mini)
10         cout << "S";
11     else
12         cout << "N";
13 }
```

Fila

```
1  #include <bits/stdc++.h>
2  using namespace std;
3  #define int long long
4  int32_t main()
5  {
6      int n;
7      cin >> n;
8      vector<int> a(n);
9
10     for (int i = 0; i < n; i++)
11     {
12         cin >> a[i];
13     }
```

```

14     int resp = 0;
15     int maior = a[n - 1];
16     for (int i = n - 2; i >= 0; i--)
17     {
18         if (a[i] <= maior)
19             resp++;
20         else
21             maior = a[i];
22     }
23     cout << resp;
24 }

```

Gráfico de Barras

```

1
2  #include <bits/stdc++.h>
3  using namespace std;
4
5  int main()
6  {
7      int n;
8      cin >> n;
9      vector<int> a(n);
10     int maior = 0;
11     for (int i = 0; i < n; i++)
12     {
13         cin >> a[i];
14         maior = max(maior, a[i]);
15     }
16     for (int j = maior; j > 0; j--)
17     {
18         for (int i = 0; i < n; i++)
19         {
20             if (a[i] >= j)
21                 cout << 1 << " ";
22             else
23                 cout << 0 << " ";
24         }
25         cout << endl;
26     }
27 }

```
